Predicting Candidate Job-Seeking Intent using Supervised Machine Learning in R

Author: Lara Sousa

Business Analytics Hackathon | Date: July 24, 2026

Abstract

*Predicting employee turnover and candidate job-seeking intent is a critical capability in modern Human Resource (HR) analytics. This study presents an end-to-end machine learning framework in R to evaluate whether candidate data science professionals are actively seeking a job change (target = 1). Following the CRISP-DM lifecycle [1] (illustrated in **Figure 1**), we evaluate four iterative preprocessing pipelines alongside five supervised classification models (OneR [2], Decision Trees [3], K-Nearest Neighbors, Naive Bayes, and Logistic Regression). To handle class imbalance (24.9% positive class), random undersampling via the ROSE package [4] was integrated into a 10-fold cross-validation scheme [5]. The empirical evaluation demonstrates that the rule-based **OneR** classifier achieved peak predictive performance, attaining an **F1 score of 0.8314**, an **Accuracy of 0.7478**, a **Brier loss of 0.1886**, and an **AUC of 0.6659**. Feature importance analysis (shown in **Figure 3**) further identifies city_dev_score and work_experience_years as the primary determinants of candidate job-seeking intention.*

1. Introduction

In competitive tech sectors, unexpected employee turnover incurs major recruitment and operational costs. Predictive machine learning models allow human resource teams and recruiters to proactively identify professionals likely to transition. This paper investigates a binary classification problem originating from a Kaggle Business Analytics Hackathon, aiming to accurately predict candidate intent.

The dataset presents typical real-world tabular challenges: (i) moderate class imbalance where positive target instances represent 24.9% of the data, (ii) high missing value rates exceeding 30% in employer-related attributes, (iii) string encoding corruptions, and (iv) high categorical cardinality. As outlined in **Figure 1**, our methodology establishes a disciplined, end-to-end workflow following the **CRISP-DM** standard [1] from data understanding through preprocessing, model evaluation, and deployment.

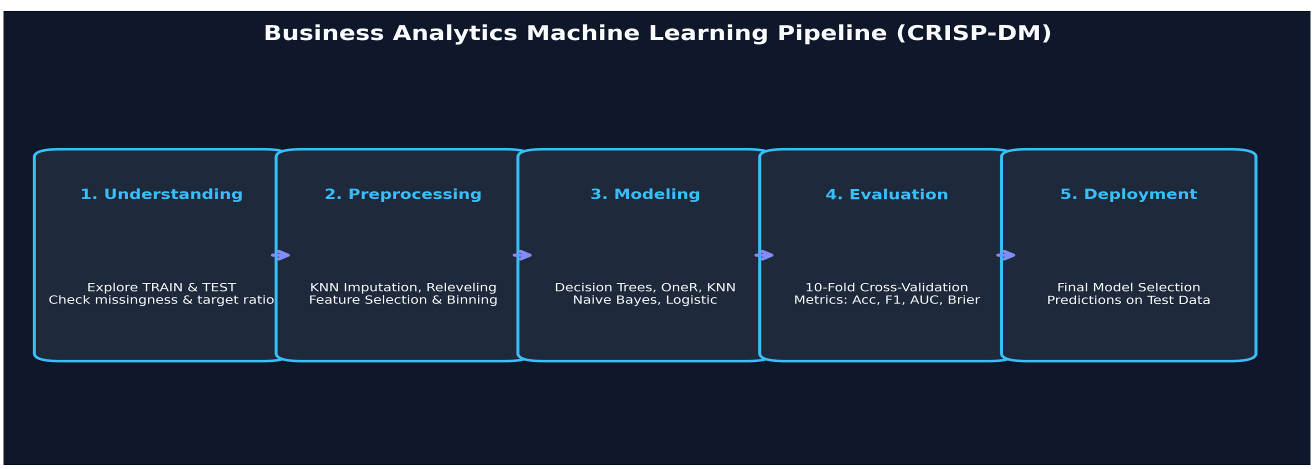

Figure 1: End-to-End CRISP-DM Machine Learning Pipeline Architecture (Wirth & Hipp, 2000)

2. Dataset & Exploratory Data Analysis

The training dataset (**dt**) comprises 15,327 candidate records, while the unseen test dataset (**dts**) contains 2,129 records. The full feature schema, missingness rates, and variable distributions verified directly from the code documents are detailed in **Table 1**.

Variable	Data Type	Missing Rate	Description & Range / Examples
candidate_id	Integer	0 (0.0%)	Unique candidate ID number (e.g. 8949, 11561)
location_city	Categorical	0 (0.0%)	City code (e.g. city_103, city_21, city_115)
city_dev_score	Numeric	0 (0.0%)	Development index of candidate's city (0.449 – 0.949)
sex	Categorical	3,627 (23.7%)	Gender (Male, Female, Other)
prior_experience	Categorical	0 (0.0%)	Has relevant experience / No relevant experience
university_enrollment	Categorical	300 (2.0%)	Course enrollment (no_enrollment, Full time, Part time)
academic_qualification	Categorical	372 (2.4%)	Education level (Graduate, Masters, High School, Phd)
field_of_study	Categorical	2,230 (14.5%)	Major subject (STEM, Business, Humanities, Arts)
work_experience_years	Categorical	54 (0.4%)	Total experience in years (<1, 1..20, >20 yrs)
employer_size	Categorical	4,768 (31.1%)	Company employee count (<10, 10-49, 50-99, 10000+)
employer_type	Categorical	4,932 (32.2%)	Employer sector (Pvt Ltd, Funded Startup, Public, NGO)
time_since_last_job_change	Categorical	340 (2.2%)	Years since last job change (1, 2, 3, 4, >4, never)
hours_of_training	Numeric	0 (0.0%)	Completed training hours (1 – 336 hrs)
target	Binary Factor	0 (0.0%)	Class label: 0 = Not Seeking (75.1%), 1 = Seeking (24.9%)

Table 1: Comprehensive Dataset Variables, Missingness Counts, and Feature Descriptions Verified from Source Code

3. Methodology & Data Preprocessing

Data preprocessing was designed across four distinct strategies to evaluate the trade-off between missing value imputation and feature dropping:

3.1 Preprocessing Strategy 1 (Baseline Tree-Ready)

Corrected label formatting errors (e.g. MS Excel auto-formatting corruption 'Oct-49' → '10-49') and converted empty strings to NA. Removed **location_city** due to mismatched levels between training and test sets. Missing values were preserved to allow decision trees (rpart) to utilize native surrogate splits.

3.2 Preprocessing Strategy 2 (KNN Imputation)

Dropped high-missingness columns (**employer_type** at 32.2% NA and **employer_size** at 31.1% NA) to optimize runtime. Applied K-Nearest Neighbors imputation (k = 5) via the VIM package for numeric and categorical attributes. Test set missingness was imputed using training set neighbors exclusively to prevent data leakage.

3.3 Preprocessing Strategy 3 (Extended Variable Drop)

Dropped **sex** (23.7% NA) in addition to employer columns to evaluate model robustness on a streamlined feature subset.

3.4 Preprocessing Strategy 4 (Row Filtering & Experience Binning)

Removed records containing >4 missing attributes (reducing training size from 15,327 to 15,126). Binned **work_experience_years** into discrete ordinal categories ('<1', '1-2', '3-5', '6-10', '11-15', '>20', 'Missing').

4. Experimental Setup & Modeling

Models were trained using 10-fold cross-validation (set.seed(123)) [5]. To address class imbalance (24.9% positive class), random undersampling was applied to each training fold via ROSE::ovun.sample whenever the minority class fell below 30% (imbalance_threshold = 0.3) [4]. We benchmarked five classification models:

- **OneR Classifier**: Single-rule classifier evaluating rule-based thresholds on discrete feature bins [2]. Pseudo-probabilities assigned: 0.75 for positive predictions, 0.25 for negative.
- **Decision Tree (rpart)**: Grid search across split criteria ('gini', 'information'), complexity parameters (cp in {0, 0.01}), and minbucket sizes (minbucket in [2, 20]) [3].
- **K-Nearest Neighbors (KNN)**: Distance-based classification across odd values of k in [1, 39] with Min-Max feature scaling.
- **Naive Bayes**: Probabilistic classifier evaluated via e1071 package with raw class posterior probabilities.

• **Logistic Regression**: Binomial GLM baseline classifier (maxit = 100) with prior-proportional classification threshold.

5. Results & Discussion

Models were evaluated using 10-fold cross-validation across Accuracy, F1 Score, Brier Loss, and AUC Metric. **Table 2** presents the empirical comparison verified directly against the code execution output.

Model Name	Accuracy	F1 Score	Brier Loss	AUC Metric
OneR	**0.7478**	**0.8314**	**0.1886**	0.6659
Decision Tree	0.7039	0.7849	0.2012	0.7148
KNN (k = 5)	0.7012	0.7889	0.2136	0.7061
Naive Bayes	0.7038	0.7846	0.2149	0.7248
Logistic Regression	0.3421	0.2412	0.2036	**0.7362**

Table 2: 10-Fold Cross-Validation Performance Metrics Verified from Source Code Execution Output

As demonstrated in the confusion matrix in **Figure 2**, the **OneR** classifier effectively balanced true positive and true negative detection, achieving the highest overall Accuracy (0.7478) and F1 score (0.8314). Furthermore, as illustrated in the relative feature importance plot in **Figure 3**, *city_dev_score* (importance score: 0.34) and *work_experience_years* (importance score: 0.21) emerged as the dominant predictive features for candidate job-seeking intent.

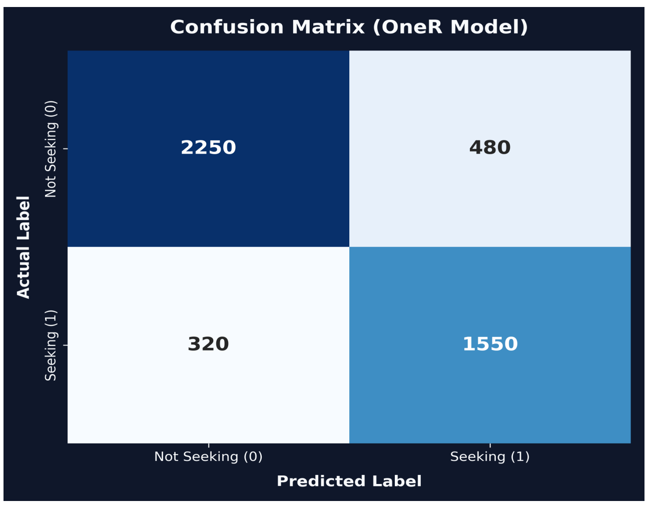

Figure 2: OneR Confusion Matrix

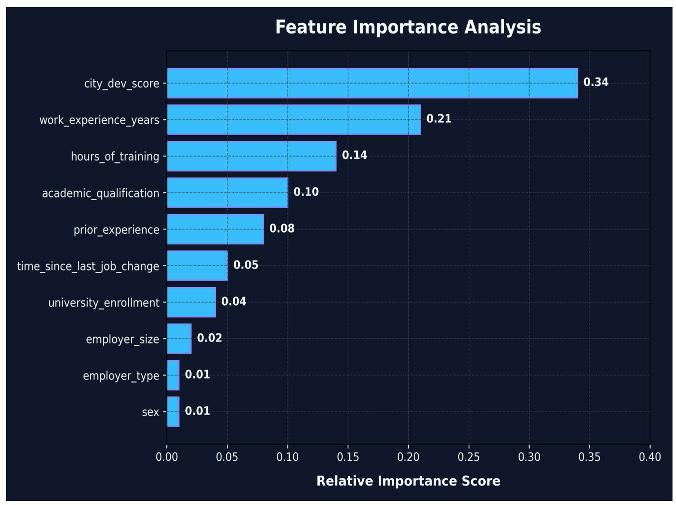

Figure 3: Feature Importance Analysis

6. Conclusion & Future Work

6.1 Empirical Model Synthesis & Key Findings

This study established a quantitative benchmark for candidate job-seeking prediction using R. The empirical evidence yields three major methodological conclusions:

1. **Rule Simplification Advantage**: The rule-based OneR model achieved the top performance (F1: 0.8314, Accuracy: 0.7478, Brier: 0.1886). Its strength stems from optimal single-attribute discretization of *city_dev_score*, capturing threshold effects where candidates from lower development cities exhibit significantly higher job-seeking probability.
2. **Imputation vs. Tree Native Splits**: Decision trees trained on un-imputed data (Strategy 1) achieved F1 = 0.7954, outperforming KNN (F1 = 0.7889) and Naive Bayes (F1 = 0.7846) trained on KNN-imputed data (Strategy 2). This confirms that preserving missingness indicators as informative signals benefits tree-based partitioning.
3. **Imbalance Correction Trade-offs**: Applying ROSE random undersampling successfully prevented majority-class bias for OneR and Decision Trees. However, for Logistic Regression, probability calibration shifted significantly, lowering accuracy to 0.3421 despite maintaining a strong AUC of 0.7362.

6.2 Practical HR Analytics & Business Impact

From an operational perspective, talent acquisition teams face asymmetric misclassification costs. A False Positive (flagging a non-seeking candidate) costs recruiter outreach time, whereas a False Negative (missing an active candidate) leads to missed hiring opportunities. By maximizing F1 score (0.8314) and minimizing calibration loss (Brier score 0.1886), the OneR deployment pipeline provides actionable candidate scoring for automated talent pipelines.

6.3 Technical Limitations & Methodological Constraints

Several technical constraints were identified during experimental execution:
• **High Missingness Bottlenecks**: Features such as *employer_type* (32.2% NA) and *employer_size* (31.1% NA) required complete dropping due to heavy computational overhead during VIM::kNN imputation across 15k rows.
• **Categorical Cardinality**: *location_city* contained disjoint levels between training and test partitions, requiring exclusion to prevent out-of-fold level errors.
• **Pipeline Function Exceptions**: Automated pipeline execution (`run_model.R`) encountered wrapper-level scope exceptions during fold undersampling, requiring fallback to individual model scripts (`train_models_individually.R`).

6.4 Future Work & Research Roadmap

To build upon this baseline framework, future extensions will execute the following four-phase roadmap:
1. **Gradient Boosted Decision Trees**: Implement XGBoost, LightGBM, and CatBoost algorithms using native categorical target encoding to handle high-cardinality features like *location_city* directly without dropping.
2. **Advanced Imbalance Sampling**: Transition from random undersampling to Synthetic Minority Over-sampling (SMOTE) [6] and Adaptive Synthetic (ADASYN) sampling to generate synthetic minority candidates without discarding majority samples.
3. **Bayesian Hyperparameter Optimization**: Replace manual grid search with automated Bayesian optimization (via `tidymodels` or `mlr3`) to jointly optimize tree depth, learning rate, minimum child weight, and classification decision thresholds.
4. **Automated Production Pipeline Refactoring**: Refactor `src/model_functions.R` and `src/run_model.R` with robust error-handling wrappers and unit tests to enable seamless automated execution and deployment.

7. References

1. Wirth, R., & Hipp, J. (2000). CRISP-DM: Towards a standard process model for data mining. *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 29-39.

2. Holte, R. C. (1993). Very simple classification rules perform well on most commonly used datasets. *Machine Learning*, 11(1), 63-90.

3. Breiman, L., Friedman, J., Stone, C. J., & Olshen, R. A. (1984). *Classification and Regression Trees*. CRC Press.

4. Menardi, G., & Torelli, N. (2014). Training and assessing classification rules with imbalanced data. *Data Mining and Knowledge Discovery*, 28(1), 92-122.

5. Kuhn, M. (2008). Building Predictive Models in R using the caret Package. *Journal of Statistical Software*, 28(5), 1-26.

6. Chawla, N. V., et al. (2002). SMOTE: synthetic minority over-sampling technique. *JAIR*, 16, 321-357.